ORCHLANG / COMPILER DESIGN REVIEW

The invoice can reveal the secret

Static checks for secret-dependent per-model usage in OrchLang

Nambi Rajan M · 24BAI0072 · Vellore Institute of Technology

MODELED COMPARISON

OBSERVER	itemized model usage
PUBLIC INPUTS	held equal
MODEL BEHAVIOR	aligned by relation
SECRET	varies

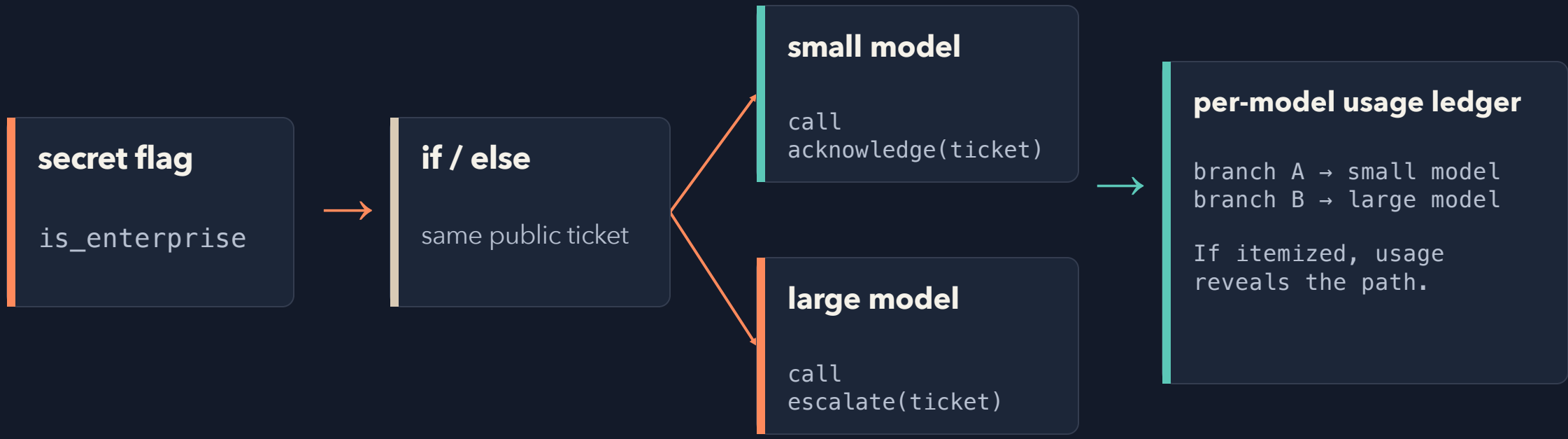
MODEL USAGE TRACE

MODEL	INVOICE ROW
fast	branch A
large	branch B

If usage is itemized, the selected path is visible.

One secret bit selects the model-call path

Its per-model usage trace may be visible to an invoice reader.



SECRET-DEPENDENT CONTROL FLOW → MODELED USAGE OBSERVATION

SOURCE docs/PAPER.md §1; examples/research/rejected/equal_bounds_different_inputs.orch

A secret branch can change model spend

The public ticket enters the prompt; the guard only chooses a call path.

SECRET CONDITION

if enterprise

selects one call path

→

prompt input

public ticket
flows to the model

ticket
→

value sinks

no secret argument
no secret output
no secret tool effect

RESOURCE OBSERVATION

The branch changes whether a model is called and what appears on the invoice.

SOURCE src/semantic_analyzer.cpp; tests/tests.cpp (flow and billing checks)

OBSERVATION

Define usage before claiming independence

Per-model usage is measured; exact request identity appears in the proof coupling.

```
Obs(e)[model] = (input, output, calls)
```

input tokens

measured per model

output tokens

measured per model

call count

measured per model

The mock model defines this trace; no live-provider invoice was measured.

SOURCE PAPER §4.1; src/interpreter.cpp:248-267; bench/results/sourced.json; bench/results/sourced_request_sensitive.json

Bounds and secrecy answer different questions

Keep one-run resource safety separate from two-run relational safety.

ONE EXECUTION

Question
How much can this workflow consume?

Claim
Cost(e) ≤ B

Evidence
Structural worst-case bound

PAIRED EXECUTIONS

Question
Can changing the secret change the bill?

Claim
$Obs(e_0) = Obs(e_1)$

Evidence
Paired-run check under declared relation

SOURCE src/cost_analyzer.cpp; src/relational.cpp; tests/tests.cpp

COUNTEREXAMPLE

Equal caps do not certify equal usage traces

A secret can choose between two fixed public values with the same declared cap.

SECRET = 0

```
fixed public value: left_source
cap = 32 tokens
realized length = n
input usage = 3 + n
shared-oracle output = o₀
```

SECRET = 1

```
fixed public value: right_source
cap = 32 tokens
realized length = m
input usage = 3 + m
shared-oracle output = o₀
```

same per-call envelope: input ≤ 35 · output ≤ 4 · calls = 1

SOURCE examples/research/rejected/equal_bounds_different_inputs.orch; src/interpreter.cpp:248-267

A shared ceiling says nothing about the realized pair

The public fixture stays fixed; only the secret changes the selected value.

BOTH ARMS HAVE THE SAME DECLARED ENVELOPE

`input ≤ 35 estimated tokens  ·  output ≤ 4 tokens  ·  calls = 1`

SECRET = 0

```
input usage = 3 + n
output = o₀  ·  default oracle at i=0
```

SECRET = 1

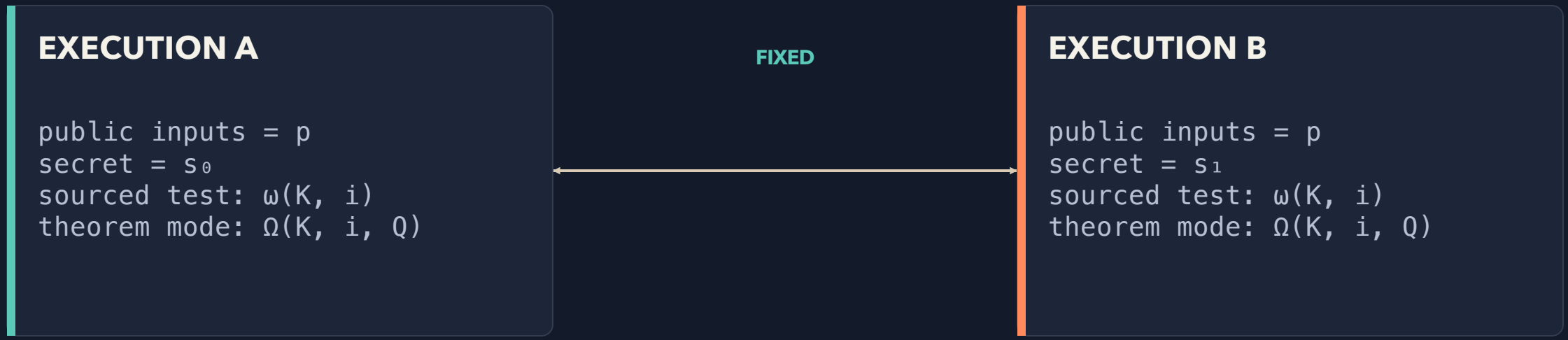
```
input usage = 3 + m
output = o₀  ·  default oracle at i=0
```

For a fixed public fixture where n ≠ m, the shared cap does not make the traces equal.

SOURCE examples/research/rejected/equal_bounds_different_inputs.orch; include/semantic_analyzer.hpp:24; src/interpreter.cpp:248-267

Pair executions under a declared relation

Sourced tests use contract/index; the theorem couples contract, index, and request.



K = resolved contract · i = per-contract call index · Q = DSL abstract request

SOURCE docs/PAPER.md §4.2; src/interpreter.cpp; bench/evaluate_sourced.py; both sourced result files

Match the DSL request to a resolved model

Exact template bytes and ordered argument relations; provider payload details stay outside scope.

ARM 0

call event

```
contract=K  ·  template="reply {ticket}"  ·  args=[outer(ticket#7)]
```

ARM 1

call event

```
contract=K  ·  template="reply {ticket}"  ·  args=[outer(ticket#7)]
```

same ordered request-and-billing structure

SOURCE docs/PAPER.md §4.3; include/relational.hpp; src/relational.cpp; src/semantic_analyzer.cpp

A secret branch passes only when its signatures align

The rule checks equality under the declared shared-oracle contract.

✓ ACCEPT

```
arm 0: Call(m, τ + |ticket|)
arm 1: Call(m, τ + |ticket|)

resolved events are equal
```

× REJECT

```
arm 0: Call(m, τ + |x|)
arm 1: Call(m, τ + |y|)

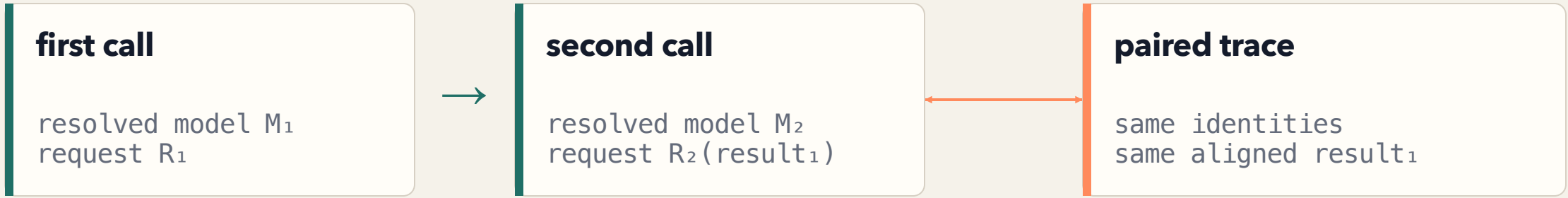
terms are not known to agree
```

Sufficient and deliberately incomplete for the declared observation.

SOURCE docs/FORMAL_CONTRACT.md §4; include/relational.hpp; src/relational.cpp; tests/tests.cpp (E236 cases)

Matched results can feed a later call

Balanced chains preserve resolved identities and per-contract call positions.



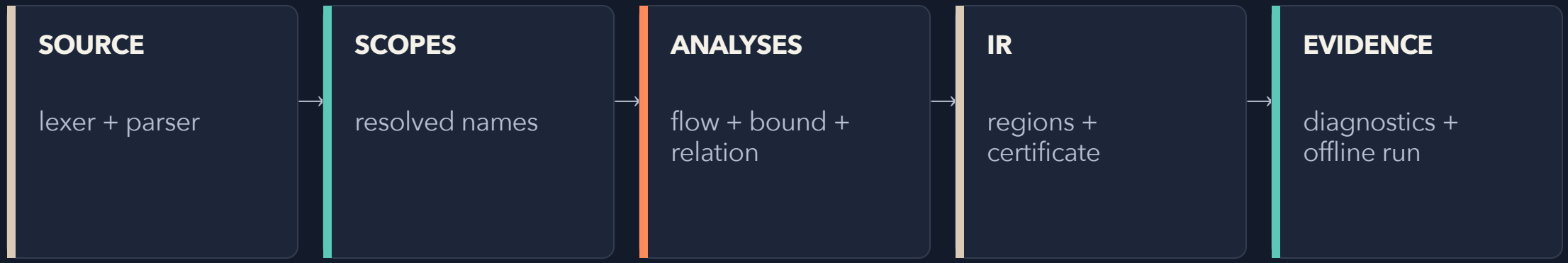
Required evidence

Final rule must explain why both second requests and their resulting bills align.

SOURCE src/relational.cpp; src/interpreter.cpp; tests/tests.cpp (matched-chain regression); E267 cross-tokenizer rejection

The guarantee lives in an ordinary compiler pipeline

From source locations to diagnostics, certificates, and offline checks.



No model key, live provider call, or network access is needed for the static checks.

SOURCE src/main.cpp; src/semantic_analyzer.cpp; src/cost_analyzer.cpp; src/relational.cpp; src/certificate.cpp; src/interpreter.cpp

A rejection should name the request contract that differs

Compiler output makes the failed comparison reviewable at source level.

SOURCE SPAN

```
  if choose {
    let result_left: text = call
ask(left_source) using shared;
  } else {
    let result_right: text = call
ask(right_source) using shared;
  }
```

COMPILER DIAGNOSTIC

```
equal_bounds_different_inputs.orch:7:3:
 error [E236] this branch is guarded by secret
data and its two arms have different billing
contracts or request content; then-arm
[mock:research-model (max_tokens=4)
request(template="Request: {t}",
args=[outer(left_source#2)])] and else-arm
[mock:research-model (max_tokens=4)
request(template="Request: {t}",
args=[outer(right_source#3)])]
```

SOURCE examples/research/rejected/equal_bounds_different_inputs.orch; final compiler stdout/stderr

Test bounds and paired traces separately

Finite experiments check implementation behavior; they do not replace a proof.

1 COMPILE
Regression suite and exact diagnostics

2 BOUND
Run generated workflows; compare each component to its bound

3 RELATION
Pair executions with fixed public inputs and aligned oracle

Finite runs validate implementation behavior; formal soundness needs a separate argument.

SOURCE Makefile; tests/tests.cpp; bench/evaluate_sourced.py; bench/generate_sourced.py

The mock-runtime results, with their denominators

Two oracle modes repeat the same offline grid; the denominators stay per mode.

15 / 15

Balanced patterns

both modes · baseline 8 / 15

0 / 2,250

Balanced pairs

mismatches in each mode

15 / 15

Mutations rejected

2,250 paired witnesses per mode

0 / 1,500

Bound violations

across base executions per mode

`30 independent source-bound reports checked · compiler SHA-256 5bae3be94e9b`

SOURCE bench/results/sourced.json; bench/results/sourced_request_sensitive.json; bench/sourced/manifest.json; bench/results/source_bound_reports/

One paired witness shows what the rejection catches

PDL-derived mutation · seed 1 · query=0 · public tool=false

ORDINAL MOCK · contract/index

```
secret=false  3/60/95
secret=true   2/57/54

calls / input / output · mock / common-
metric-worker / cap 96
```

REQUEST-SENSITIVE · contract/index/Q

```
secret=false  3/20/108
secret=true   2/17/71

calls / input / output · mock / common-
metric-worker / cap 96
```

SAME PUBLIC FIXTURE · 2,250/2,250 witnesses per mode · repeated grids are not extra patterns

SOURCE bench/results/sourced.json; bench/results/sourced_request_sensitive.json; bench/evaluate_sourced.py

The contribution sits beside established foundations

Resource-aware IFC, relational cost reasoning, and LLM side-channel studies all matter here.

RESOURCE-AWARE IFC

Ngo et al. · IEEE S&P 2017
Resource behavior can be part of a noninterference guarantee.

RELATIONAL COST

Çiçek et al. · POPL 2017
Relational reasoning bounds cost differences between runs.

PROBABILISTIC COST

Ngo et al. · PLDI 2018
Expected resource bounds for probabilistic programs.

PROBABILISTIC LLM FLOWS

Mandel et al. · ICML 2026
PPDL propagates uncertainty across LLM and tool calls.

QUANTITATIVE SIDE CHANNELS

Basin & Köpf · CCS 2007
Information-theoretic analysis of adaptive observations.

LLM LENGTH CHANNELS

Weiss et al. · 2024; Zhang et al. · 2024
Token length and timing can expose information.

OrchLang's scope: a sufficient compiler check for its declared bill observation and coupling.

SOURCE Sources [1]–[7] in sources.md

A useful check begins with a narrow, explicit contract

State the modeled request/oracle envelope, evidence, and proof work still open.

SUPPORTED ENVELOPE

Local structural signature check
Unary token bounds under declared caps
Matched two-call alignment for compatible contracts

OPEN / ASSUMPTIONS

Certificate withholds workflow claim on declassification
Full provider payload and real tokenizers
Incomplete rule; three mock precision probes

A useful guarantee starts by naming exactly what is observed and compared.

SOURCE docs/PRIOR_ART_MATRIX.md; docs/RESEARCH_REVIEW_2026-10-09.md; sources.md

Formal observation and paired-run relation

Observation function

Per-model input tokens, output tokens, and call count.

Public-input equality

Corresponding public bindings carry equal values.

Secret variation

The compared runs may differ in secret bindings.

Oracle coupling

Theorem: Ω(contract, index, DSL request); both modes use the same grid.

Theorem and limits

No declassification; pen-and-paper proof sketch is not mechanized.

SOURCE docs/PAPER.md; src/interpreter.cpp; bench/evaluate_sourced.py

Signature grammar and transfer rules

Resolved model contract

Provider, model, output cap, and configured unit price.

Exact template and arguments

Decoded template bytes plus ordered literal/value atoms.

Outer / prior-result identity

Outer bindings compare by identity; earlier results by position.

Default and opt-in oracle

Default key is contract/index; opt-in adds framed request identity.

Branch transfer and release scope

Local check only; certificate withholds claim on declassification.

SOURCE include/semantic_analyzer.hpp; include/relational.hpp; src/relational.cpp

Unary bounds and their assumptions

Output cap guarantee

Output tokens are bounded by each configured provider/model cap.

Prompt/input estimate

Input bound assumes the additive estimate covers the complete request.

Tokenizer parameter

The selected --chars-per-token value is recorded in the certificate.

Retry multiplier

A literal bounded retry scales the body bound.

Monetary reporting

Prices are reported only if configured; token bounds are the guarantee.

SOURCE include/semantic_analyzer.hpp; src/semantic_analyzer.cpp; src/cost_analyzer.cpp; src/certificate.cpp

Evaluation protocol and reproducibility

Compiler source

compiler SHA-256 5bae3be94e9b · baseline a49fcdbd

Validation run

make check · 111 C++ regressions · 19 auditor tests

Sourced benchmark grid

15 examples · 45 variants · 1,500 base runs · 30 audited reports

Paired fixture

25 seeds × lengths 0/37/160 × 2 public flags × 2 secrets

Oracle and scope

Ordinal and request-sensitive modes; same grids, not summed.

SOURCE audit/final/manifest.json; audit/final/auditor-tests.log; bench/sourced/manifest.json; bench/results/sourced.json; bench/results/sourced_request_sensitive.json; bench/evaluate_sourced.py

Diagnostics and representative edge cases

Mismatched resolved model

E236 identifies unequal resolved contracts or request signatures.

Mismatched request term

E267 rejects chains across incompatible model/tokenizer identities.

Secret request value

E230 rejects a direct secret request value.

Unsupported chained alignment

Use a source-located compiler diagnostic from the hash-matched run.

Untrusted spend warning

W237 warns when untrusted control influences modeled spend.

SOURCE src/diagnostic.cpp; src/semantic_analyzer.cpp; src/relational.cpp; tests/tests.cpp

Likely examiner questions

Why not compare upper bounds?	A maximum does not establish equality between realized costs.
What is the observer?	State the implemented per-model observation; do not claim timing or network coverage.
What aligns stochastic calls?	State the exact shared-oracle/request identity relation and assumptions.
Is the check complete?	No. Give the final accepted/rejected boundary example.
Is token-cost leakage new?	No. Position the compiler application beside established prior work.
What is proved versus tested?	Separate the soundness argument from finite implementation experiments.

SOURCE docs/PAPER.md; docs/PRIOR_ART_MATRIX.md; bench/results/sourced.json; bench/results/sourced_request_sensitive.json; sources.md

Adjacent LLM and agent checks target different properties

The neighboring work strengthens the framing; it does not establish this billing relation.

APPL · ACL 2025

Prompt language and parallel/asynchronous runtime; tracing and replay.

PPDL · ICML 2026

Probabilistic LLM flows; uncertainty propagates across calls and tools.

CaMeL · preprint

Separates trusted control/data flow and constrains tool capabilities.

Fides · preprint

Information-flow labels and policy enforcement for AI agents.

Agentproof · preprint

Static workflow-graph checks and policy-trace verification.

AgentFlow · preprint

Flow policies, bounded verification, and runtime monitoring.

These systems address workflow, uncertainty, and agent security. OrchLang checks equality of modeled bill traces.

SOURCE APPL ACL 2025; CaMeL, Fides, Agentproof, AgentFlow are preprints as of 2026-10-09

References and the updated novelty rubric

CORE REFERENCES

[1] Ngo et al. · IEEE S&P 2017 · resource-aware noninterference · DOI 10.1109/SP.2017.53

[2] Çiçek et al. · POPL 2017 · Relational Cost Analysis · DOI 10.1145/3009837.3009858

[3] Ngo et al. · PLDI 2018 · Bounded Expectations · DOI 10.1145/3192366.3192394

[4] Mandel et al. · ICML 2026 · PPDL · PMLR 306:86172–86197

[5] Basin & Köpf · CCS 2007 · quantitative side channels · DOI 10.1145/1315245.1315282

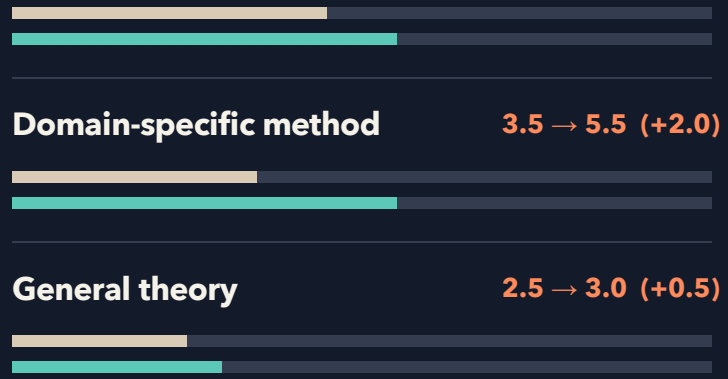
[6] Weiss et al. · USENIX Security 2024 · token-length inference

[7] Zhang et al. · Time Will Tell · arXiv:2412.15431 (preprint)

[8] Dong et al. · ACL 2025 · APPL · DOI 10.18653/v1/2025.acl-long.63

NOVELTY RUBRIC · 0–10

5 = focused contribution from established ideas
10 = major field-level advance · not acceptance odds



Scores reflect a narrower applied claim; theory stays modest.

SOURCE sources.md; docs/RESEARCH_REVIEW_2026-10-09.md; docs/PRIOR_ART_MATRIX.md; LICENSE; CITATION.cff; upstream notices